Implementation and Ramifications of TAMER-OP: a Toolkit for Algebraic Module Encodings over $\mathbb{R}^n$ and Other Posets

Erik Mendes Novak

March 10, 2026

Contents

1 Introduction **2**

2 Multiparameter persistence and its difficulties **3**
2.1 Posets, filtrations, and persistence modules . . . 3
2.2 The one-parameter scope . . . 4
2.3 What changes in several parameters . . . 4
2.4 What one computes in place of a barcode . . . 5

3 Tameness, finite encodings, and computability **6**
3.1 Finite encodings . . . 6
3.2 Tame modules and tame morphisms . . . 7
3.3 Upsets, downsets, and fringe presentations . . . 7
3.4 Why encoding is the right answer . . . 8

4 Overview of TAMER-OP as an encoding-centered system **8**
4.1 The main pipeline . . . 9
4.2 Architectural layering . . . 10

5 Pre-encoding objects and data interfaces **11**
5.1 Finite-poset presentations: finite fringe presentations and indicator data . . . 11
5.2 $\mathbb{Z}^n$ front end: flange presentations and finitely determined data . . . 12
5.3 $\mathbb{R}^n$ front-end: PL fringes and box presentations . . . 13
5.4 Raw data, filtrations, and graded complexes . . . 14

6 Bibliography notes **14**

1 Introduction

Persistent homology is most useful when it converts filtered geometric data into algebraic objects that are simultaneously expressive and computable. In the one-parameter setting these two sides align particularly well. A filtration indexed by a total order gives rise to a persistence module whose structure is rigid enough to admit interval-type decompositions, barcode summaries, and stable diagrammatic invariants [ELZ02, CEH07, CB15, EH10]. In several parameters, however, this balance breaks down. The indexing poset is no longer a chain, a barcode classification is no longer possible in general, and the cost of direct algebraic manipulation grows quickly with the size and geometry of the parameter space [CZ09, Les15, CDSGO16, Oud15]. Multiparameter persistence therefore forces a design choice: one can either work primarily with summary invariants adapted to specific tasks, or one can try to preserve a richer internal algebraic model on which broader constructions remain possible.

The software library TAMER-OP is specifically concerned with the second viewpoint. Its central principle is that the decisive computational move is not a particular invariant, nor a particular data structure for one family of modules, but rather the explicit construction of a finite encoding. Starting from raw data, finite-poset presentations, flange presentations over $\mathbb{Z}^n$, or piecewise-linear presentations over $\mathbb{R}^n$, TAMER-OP first compresses the ambient parameter poset into a finite encoding poset P together with a classifier or encoding map π. The original object is then represented by a finite-poset module, together with pushed-down presentation data on P. Once this finite internal model has been built, the rest of the library's toolkit becomes uniformly available: module morphisms, kernels and cokernels, indicator resolutions, derived functors, rank- and slice-based invariants, serialization, featurization, and visualization can all be performed against the same encoded object. In this sense, encoding is not merely preprocessing, but indeed is the organizing principle of the entire system.

This viewpoint is directly motivated by Miller's theory of tame modules over posets [Mil20]. Miller's framework shows that, under appropriate finiteness hypotheses, several apparently different descriptions of a module are equivalent: finite encodings, indicator presentations by upsets and downsets, and indicator resolutions all express the same controlled form of tameness. That equivalence extends from the theoretical to a strong implementation consequence. Rather than building separate pipelines for geometric data, algebraic summaries, and downstream statistics, one can work to make the encoding step explicit and robust, and then treat the encoded module as the shared internal object from which all later computations are derived.

TAMER-OP consequently accordingly advances three intertwined claims. Firstly, the finite-encoding perspective provides a mathematically coherent bridge between large ambient parameter spaces and finite combinatorial models. Secondly, this bridge is sufficiently explicit to support a unified software architecture that begins with multiple front end representations and data-ingestion pipelines, and ends with exact algebraic, invariant, and statistical workflows. Thirdly, this architecture is practical, as it supports tractable computation on data sets of ordinary scientific scale and is computationally competitive against existing multiparameter persistence libraries where there is implementation overlap.

In the remainder of this thesis, we first introduce the mathematical machinery behind multiparameter persistence, enough to understand what its difficulties are. Then, we show how tame modules present a computational solution. Then, we overview TAMER-OP, from the front end input, to the encoding step, to the downstream computations that are made possible due to it. Finally, we benchmark the library to demonstrate its tractability. This order is deliberate; we hope to demonstrate that nearly every distinctive feature of the library is best understood as more than just an isolated implementation trick, and indeed as a consequence of making the finite encoding step explicit and central.

2 Multiparameter persistence and its difficulties

Persistent homology begins with a filtered topological object and produces algebra. One chooses an indexing poset, usually a subset of $\mathbb{R}^d$ or $\mathbb{Z}^d$ with the coordinatewise order, and records a vector space at each parameter value together with linear maps along the order relation. In one parameter, this construction leads to a theory with a very strong structure theorem. In several parameters, the same definition still makes sense, but the structure theory changes completely. The purpose of this section is to introduce the basic language carefully and to explain why the multiparameter case resists the sort of complete description that is available in one parameter. Throughout, let $\Bbbk$ be a field.

2.1 Posets, filtrations, and persistence modules

The correct indexing objects in persistence theory are partially ordered sets. A poset may be viewed as a category, and a persistence module is then just a functor out of that category.

Definition 1 (Poset)**.** A *partially ordered set* is a set Q together with a relation $\leq$ that is reflexive, antisymmetric, and transitive.

Definition 2 (A poset as a category)**.** Given a poset $(Q, \leq)$, one forms a small category, again denoted Q, whose objects are the elements of Q, and such that there is a unique morphism $q \to q'$ if and only if $q \leq q'$. Composition is forced by transitivity and identity morphisms are forced by reflexivity.

A category of this form is thin: between any two objects there is at most one morphism. This observation is what allows order theory and homological algebra to interact in persistence.

Definition 3 (Q-module, or persistence module)**.** Let Q be a poset and let $\mathbf{Vect}_{\Bbbk}$ denote the category of vector spaces over $\Bbbk$. A *Q-module* is a covariant functor

$$M : Q \longrightarrow \mathbf{Vect}_{\Bbbk}.$$

Equivalently, one specifies a vector space $M(q)$ for each $q \in Q$ and, for each comparable pair $q \leq q'$, a linear map

$$M(q \leq q') : M(q) \to M(q')$$

subject to the functoriality relations

$$M(q \leq q) = \mathrm{id}_{M(q)}, \qquad M(q \leq q'') = M(q' \leq q'') \circ M(q \leq q')$$

whenever $q \leq q' \leq q''$.

A morphism $f : M \to N$ of Q-modules is a natural transformation. Concretely, this means that for each $q \in Q$ there is a linear map $f_q : M(q) \to N(q)$ and, for each $q \leq q'$, the square

$$\begin{tikzcd} M(q) \arrow[r, "M(q\leq q')"] \arrow[d, "f_q"'] & M(q') \arrow[d, "f_{q'}"] \\ N(q) \arrow[r, "N(q\leq q')"] & N(q') \end{tikzcd}$$

commutes.

The persistent homology modules arising from filtered topology fit into this framework in the most direct possible way: if $\mathcal{F} : Q \to \mathbf{Top}$ is a filtration, then for each homological degree i one obtains a Q-module

$$H_i(\mathcal{F}; \Bbbk) : Q \longrightarrow \mathbf{Vect}_{\Bbbk}$$

by functoriality of homology.

The category of Q-modules inherits the basic exact structure of $\mathbf{Vect}_{\Bbbk}$.

Proposition 4 (Pointwise abelian structure)**.** *For any small category $\mathcal{C}$, the functor category* $\mathrm{Fun}(\mathcal{C}, \mathbf{Vect}_{\Bbbk})$ *is abelian. In particular, when $\mathcal{C} = Q$ is a poset, kernels, cokernels, images, and coimages are computed pointwise [Mac98, Wei94].*

This pointwise abelian structure is the key to computational feasibility. If the indexing poset is finite, then a persistence module is represented by finitely many vector spaces and finitely many structure maps, so abelian constructions reduce to finite linear algebra. If the indexing poset is infinite, the same proposition still holds formally, but it does not provide a usable algorithm. This gap between formal exactness and finite computability is the problem that motivates the encoding viewpoint later in the thesis.

2.2 The one-parameter scope

The one-parameter theory is the model initial case. Let Q be $\mathbb{R}$ or $\mathbb{Z}$ with the usual total order. Then persistent homology produces modules indexed by a chain, and that special geometry forces a rigid structure theory.

Definition 5 (Interval module)**.** Let $I \subseteq \mathbb{R}$ be an interval. The associated *interval module* $\Bbbk_I$ is defined by

$$\Bbbk_I(t) = \begin{cases} \Bbbk, & t \in I, \\ 0, & t \notin I, \end{cases}$$

with structure maps equal to the identity whenever both source and target lie in I, and equal to the zero map otherwise.

Theorem 6 (Barcode decomposition)**.** *Under the usual pointwise-finite and local-finiteness hypotheses from persistence theory, a one-parameter persistence module decomposes as a direct sum of interval modules, uniquely up to isomorphism and reordering of summands [CB15, Oud15].*

This theorem explains why barcodes are so effective. In the one-parameter setting they are more than just heuristic pictures, indeed encoding the module itself under the standard finiteness assumptions. Stability theory then shows that the associated persistence diagrams vary continuously under perturbation of the filtered data [ELZ02, CEH07, CDSGO16]. Thus one-parameter persistence enjoys both classification and stability.

That combination does not generalize. It is natural to ask for the same story in several parameters, but already the algebra of the indexing category becomes too rich.

2.3 What changes in several parameters

For $d \geq 2$, the standard indexing posets are $\mathbb{Z}^d$ or $\mathbb{R}^d$ with the coordinatewise order

$$a \leq b \quad \Longleftrightarrow \quad a_i \leq b_i \text{ for all } i.$$

This change from a chain to a partial order has major consequences. For example, there are now many incomparable pairs. A module is no longer controlled by a single linear progression of births and deaths. Additionally, if one works over $\mathbb{Z}^d$ or $\mathbb{N}^d$, the algebra is equivalent to the algebra of multigraded modules over a polynomial ring.

Proposition 7 (Algebraic model over $\mathbb{N}^d$)**.** *A persistence module indexed by $\mathbb{N}^d$ is equivalent to an $\mathbb{N}^d$-graded module over the polynomial ring $\Bbbk[x_1, \ldots, x_d]$, where multiplication by x_i records the structure map in the ith coordinate direction [CZ09, Oud15].*

This equivalence is powerful, because it brings in multigraded commutative algebra. It is also, however, the source of the difficulty. The polynomial ring in several variables is not a principal ideal domain, multigraded modules need not decompose into interval modules, and indecomposables occur in families rather than as a discrete list. In particular, the multiparameter theory does not admit a barcode theorem comparable to the one-parameter case [CZ09, Les15].

So the obstruction is not just that computations become larger. The problem is also structural: in one parameter, the module itself can be packaged into a complete discrete invariant; but, in several parameters, there is in general no analogous finite complete invariant of that simple form.

This has two immediate consequences for computation.

(i) One should not expect a universal normal form that is both complete and as easy to manipulate as a barcode.

(ii) If one wants to preserve the full algebraic object, rather than only a summary of it, then one needs a different notion of finiteness.

That second point is the central issue addressed by TAMER-OP. The relevant question is not how to force the multiparameter theory back into one-parameter language, but how to represent the module by finite, computable data without dismissing the algebra one later wants to use.

2.4 What one computes in place of a barcode

Although there is no complete barcode-like invariant in general, several families of invariants remain useful. They are partial by necessity, each only recording one aspect of the module rather than all of it. We now overview some main ones.

Definition 8 (Rank invariant)**.** Let M be a Q-module. For each comparable pair $a \leq b$ in Q, the *rank invariant* is

$$\rho_M(a, b) = \operatorname{rank}\big(M(a \leq b)\big).$$

The rank invariant records how much information survives from one parameter value to another. In one parameter it determines the barcode. In several parameters it remains important, but is no longer a complete characterization (different modules may have the same rank invariant).

A second common idea is to probe a multiparameter module along one-dimensional subposets.

Definition 9 (Restriction to a chain or slice)**.** If $\lambda : Q' \to Q$ is a monotone map, then the pullback

$$\lambda^* M := M \circ \lambda$$

is a Q'-module. When Q' is totally ordered, $\lambda^* M$ is an ordinary one-parameter persistence module. Its barcode is then called a *slice barcode* or *fibered barcode*, depending on the context.

This recovers the successful one-parameter theory on each slice, and it is the basis of tools such as RIVET [LW15, RIVET]. But again, different multiparameter modules can have the same family of one-dimensional restrictions.

There are also dimension-based invariants. The most basic one is the restricted Hilbert function

$$q \longmapsto \dim_{\Bbbk} M(q).$$

This records the size of each fiber but not the structure maps between fibers.

A related family comes from chain complexes rather than from a single module. If $C^\bullet$ is a chain or cochain complex of modules on a finite encoding poset, one may define the Euler characteristic function

$$\chi_C(q) = \sum_t (-1)^t \dim_{\Bbbk} C^t(q).$$

When $C^\bullet$ arises from filtered topology, this agrees with the alternating sum of the homology dimensions at q. In this thesis, when we speak of *Euler invariants*, we mean invariants built from this Euler characteristic function, together with the finite-grid signed measures obtained from it by Mobius inversion. These invariants are useful because they are cheap to evaluate, easy to visualize, and often effective in statistical pipelines, even though they are far from complete.

The essential point is that none of these constructions replaces the module itself. Rank invariants, slice barcodes, Hilbert functions, and Euler characteristic functions are all informative, but each forgets some part of the structure. This is why the central focus of TAMER-OP is not the design of a single invariant, but rather the construction of a finite algebraic model from which many different invariants and constructions can be extracted.

3 Tameness, finite encodings, and computability

The correct response to the loss of barcode classification is to identify a class of modules that still admit finite descriptions and remain amenable to exact algebraic computation. In this thesis that role is played by tame modules in the sense of Miller [Mil20]. We now explore what finite encodings are and why tameness is so important.

3.1 Finite encodings

The basic finiteness notion is that a module factors through a finite poset.

Definition 10 (Finite encoding)**.** Let Q be a poset and let M be a Q-module. A *finite encoding* of M consists of

(i) a finite poset P,

(ii) a monotone map $\pi : Q \to P$, and

(iii) a P-module H with finite-dimensional stalks,

such that

$$M \cong \pi^* H = H \circ \pi.$$

Here π^* denotes pullback along the poset morphism π.

Definition 11 (Pullback along a poset morphism)**.** If $\pi : Q \to P$ is monotone and H is a P-module, the *pullback* of H along π is the Q-module

$$\pi^* H := H \circ \pi.$$

Thus

$$(\pi^* H)(q) = H(\pi(q))$$

for each $q \in Q$, and the structure map from q to q' is the structure map of H from $\pi(q)$ to $\pi(q')$.

Proposition 12 (Exactness of pullback)**.** *For any monotone map $\pi : Q \to P$, the pullback functor*

$$\pi^* : \mathbf{Vect}_{\Bbbk}^{P} \longrightarrow \mathbf{Vect}_{\Bbbk}^{Q}$$

is exact.

Proof. Kernels, cokernels, and finite direct sums in a functor category are computed pointwise. Pullback by precomposition simply evaluates the P-module at $\pi(q)$ instead of at q, so exactness is preserved at every stalk. □

A finite encoding is therefore not a lossy statistic. It is an isomorphic re-expression of the module through a finite index category. If $M \cong \pi^* H$, then the algebraic behavior of M is constant on each fiber of π, and every fiber and structure map of M is recovered from the finite-poset module H together with the map π.

3.2 Tame modules and tame morphisms

With the notion of finite encoding in hand, tameness is simple to state.

Definition 13 (Tame module)**.** Let Q be a poset. A Q-module M is *tame* if it admits a finite encoding.

This definition is stronger than merely asking for finite-dimensional fibers or finite-rank structure maps. It requires the entire module to be controlled by finitely many regions of parameter space. That extra strength is exactly what makes it suitable for computation.

There is, however, a subtle categorical point. If one allows all natural transformations between tame modules, kernels and cokernels need not remain finitely encoded. The correct category is obtained by restricting to morphisms that are themselves compatible with a common finite description.

Definition 14 (Tame morphism)**.** Let M and N be tame Q-modules. A morphism $f : M \to N$ is called *tame* if there exists a common finite constant subdivision, or equivalently a common finite encoding context, in which the component maps of f are constant on each region in the obvious sense [Mil20].

Theorem 15 (Miller)**.** *With tame modules as objects and tame morphisms as morphisms, the resulting category is abelian [Mil20].*

This addresses a natural concern. One might worry that finite encodability is too restrictive to support genuine homological algebra. The point of Miller's framework is that this is not the case: the correct category of tame modules is still abelian, so kernels, cokernels, exact sequences, and derived constructions remain available. Recent work of Waas further clarifies that categories of finitely encodable persistence modules have better exact behavior than one might first expect, reinforcing the view that finite encodability is a mathematically natural rather than ad hoc hypothesis [Waa24].

Thus, the word tame should not be read as a purely artificial restriction imposed for software convenience. It identifies the point at which an ambient persistence module can be represented by finite data without collapsing to a summary invariant that loses information. For the purposes of TAMER-OP, tameness is the hypothesis that makes it possible to keep the module, rather than discard it, and still work computationally.

3.3 Upsets, downsets, and fringe presentations

Finite encodings are one language for tameness. Another is given by indicator modules attached to order-theoretic regions.

Definition 16 (Upsets and downsets)**.** Let Q be a poset.

(i) An *upset* is a subset $U \subseteq Q$ such that $q \in U$ and $q \leq q'$ imply $q' \in U$.

(ii) A *downset* is a subset $D \subseteq Q$ such that $q' \in D$ and $q \leq q'$ imply $q \in D$.

For $q \in Q$, the corresponding principal upset and principal downset are

$$\uparrow q = \{q' \in Q : q \leq q'\}, \qquad \downarrow q = \{q' \in Q : q' \leq q\}.$$

Definition 17 (Indicator modules)**.** To an upset $U \subseteq Q$ one associates the *upset indicator module* $\Bbbk\{U\}$, defined by

$$\Bbbk\{U\}(q) = \begin{cases} \Bbbk, & q \in U, \\ 0, & q \notin U, \end{cases}$$

with the evident structure maps. Dually, a downset D determines a *downset indicator module* $\Bbbk\{D\}$.

On a finite poset, principal upset indicators play the role of indecomposable projective modules, and principal downset indicators play the role of indecomposable injective modules. This is why they are the correct basic pieces for exact algebra on finite posets.

Definition 18 (Finite fringe presentation)**.** A *finite fringe presentation* of a Q-module M consists of a morphism

$$\varphi : \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\}$$

from a finite direct sum of upset indicator modules to a finite direct sum of downset indicator modules such that $M \cong \mathrm{im}(\varphi)$ [Mil20].

This presentation language is one of the key bridges between geometry and algebra in the library. Upsets encode where generators are born and persist upward in the order; downsets encode where relations become enforced; and the matrix of φ records how these pieces interact.

3.4 Why encoding is the right answer

The key fact is that tameness admits several equivalent finite descriptions.

Theorem 19 (Syzygy theorem, after Miller)**.** *For a module over a poset, the following finiteness conditions are equivalent: existence of a finite encoding, existence of a finite fringe presentation, and existence of finite resolutions by indicator modules [Mil20].*

This theorem explains why encoding is the right computational answer to the difficulties of multiparameter persistence. We describe three consequences of it now.

Firstly, it says that one does not need to choose between a geometric region picture, a presentation by births and deaths, and a homological description by resolutions. For tame modules, these are different finite views of the same object.

Secondly, it explains what is gained by encoding. We do not package a multiparameter module back into one clean barcode, because that is impossible in general. Instead, we pass to a finite poset on which the module is represented exactly by finite data. Once that finite representation has been constructed, one can carry out actual algebra: kernels, cokernels, change-of-poset operations, indicator resolutions, and derived functor calculations. Invariants such as rank functions, slice barcodes, Hilbert functions, Euler characteristic functions, and their associated signed measures are then extracted from this finite model, rather than substituted for it.

Thirdly, it explains why this approach does not lose information in the sense relevant to the library. The encoded module on the finite poset is not a secondary summary attached to the original module, but rather the finite algebraic object through which the original module is being represented. The purpose of the encoding step is therefore not compression for its own sake. It is to move from an ambient indexing space that is too large for direct exact computation to a finite model on which exact computation is possible.

This is the perspective adopted throughout the rest of the thesis. The next section therefore begins with the architecture of the encoding-centered system itself, because once the encoded finite-poset module has been built, the rest of the library follows from that choice.

4 Overview of TAMER-OP as an encoding-centered system

The previous section explained why finite encodings are mathematically legitimate and computationally necessary. This section explains how that idea informs the organization of TAMER-OP. We identify the stable mathematical interfaces that the implementation is built around; these are arranged so that many front-end objects feed a single encoding step, and that encoding step in turn feeds many downstream computations.

4.1 The main pipeline

At the highest level, the library is organized around the composite workflow

$$\mathcal{I} \xrightarrow{\text{prepare}} \mathcal{P} \xrightarrow{\text{encode}} \mathcal{E} \xrightarrow{\text{analyze}} \mathcal{O},$$

where:

- $\mathcal{I}$ denotes raw inputs, such as data files, point clouds, graphs, images, or already structured presentation objects;
- $\mathcal{P}$ denotes pre-encoding mathematical objects, such as finite fringe modules, flanges over $\mathbb{Z}^n$, PL or box presentations over $\mathbb{R}^n$, and graded complexes produced by ingestion;
- $\mathcal{E}$ denotes finite encodings, represented concretely by a finite poset together with an encoded module and the maps needed to interpret it back in the ambient parameter space; and
- $\mathcal{O}$ denotes downstream outputs, including categorical constructions, derived functors, invariants, feature vectors, serialized artifacts, and visual objects.

The crucial middle object $\mathcal{E}$ is what the codebase packages as an `EncodingResult`. Stripping away software metadata, the mathematical content of such an object is the tuple

$$(P, M, \pi, H).$$

Here P is a finite encoding poset, M is a finite-poset module on P, π is the classifier or encoding map from the ambient domain to P, and H is a pushed-down fringe presentation when such a presentation is available. Not every later calculation uses all four pieces, but together they constitute the canonical object available to the rest of the system.

This packaging is contextually important: the encoding step does not merely return a poset and forget how it arose; it returns a finite module, a mechanism for point location in the original parameter space, and enough provenance to support comparison, serialization, and visualization. From the user-facing point of view, this is the moment at which a raw front end object becomes a reusable algebraic object.

Figure 1 summarizes the resulting architecture.

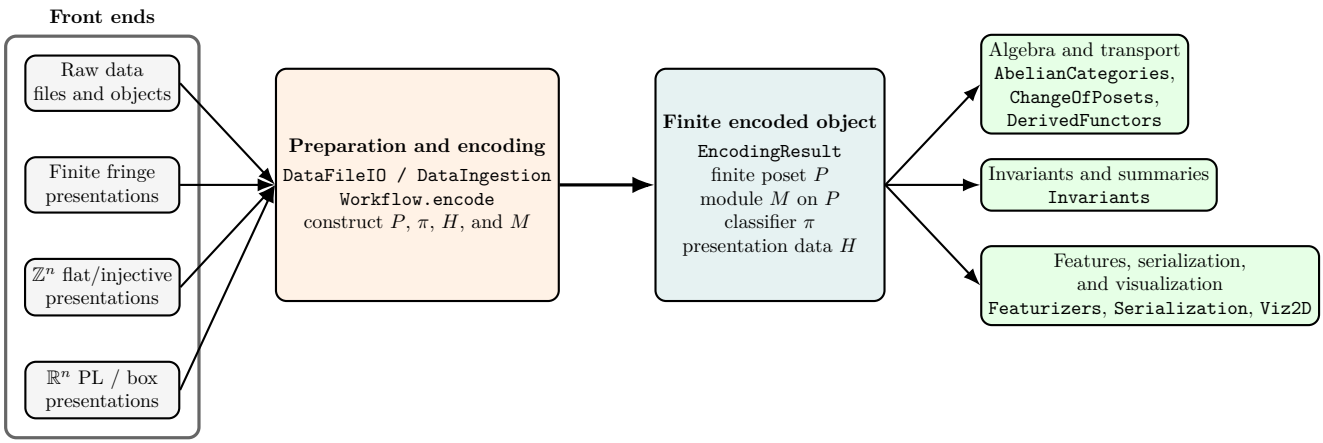

Figure 1: A system-level view of TAMER-OP. Multiple front ends are normalized into a single encoded object, and all later computation branches from that encoded object rather than from the original ambient representation.

The diagram is purposefully thin in the center but branched in input and output. The left side is diverse because the library is intended to accept front-end objects in the form most natural

to their source. The middle is narrow because there is only one conceptual move that matters: build a finite encoding. The right side branches again because, once the encoded object exists, many different computational goals can be pursued from the same internal representation.

This thin diagrammatic section characterizing encoding identifies the precise place at which the computational problem changes style. Before encoding, one is still dealing with ambient parameter spaces, geometric representations, filtration choices, and presentation-specific subtleties. After encoding, one is dealing with a finite-poset module. The post-encoding world is therefore uniform even though the pre-encoding world is intentionally heterogeneous.

This is also the main mathematical reason that TAMER-OP can do things that are awkward or impossible for existing libraries, which are generally built around architectures that prioritize only summaries and invariants. Once an actual encoded module has been retained, one may compute kernels, cokernels, equalizers, pushouts, projective and injective approximations, derived functors, and comparison transports before one ever decides which invariant or feature vector to extract. The encoded module is therefore not just a transient intermediate artifact, but the central object around which the rest of the pipeline is organized.

4.2 Architectural layering

The flow chart of figure 1 represents the curated user experience, but to orchestrate that in TAMER-OP we designed a layered architecture that separates code files based on use, with each layer building on the abstractions introduced by the previous one.

At the bottom lies a runtime and linear-algebra layer. The role of this layer is to provide exact and backend-aware matrix kernels, together with caches and autotuned thresholds, so that later algebraic code can be written at the level of modules rather than at the level of ad hoc numerical special cases. In the present codebase this role is played chiefly by the `CoreModules` and `FieldLinAlg` file families.

Above that lies the finite-poset representation layer. This is the layer in which finite posets, upsets, downsets, fringe modules, finite-poset modules, abelian-category constructions, and indicator resolutions are represented concretely. Conceptually, this is the algebraic core of the system. Once an object is being expressed by this layer, ordinary linear algebra and functor algebra can be applied directly. The files `FiniteFringe`, `Modules`, `AbelianCategories`, and `IndicatorResolutions` occupy this level.

The encoding layer sits above the representation layer and below the user-facing workflow. Its job is to build finite region posets and classifier maps from front-end objects over $\mathbb{Z}^n$ or $\mathbb{R}^n$, together with any compiled or cached structures needed for efficient point location and pushforward. This is where the library's main differentiating algorithms live. In the current codebase, the relevant file families are `EncodingCore`, `ZnEncoding`, `PLBackend`, and `PLPolyhedra`.

Then comes the orchestration layer. Rather than exposing every internal constructor directly to ordinary users, TAMER-OP organizes common workflows through a small number of stable entry points. The key example is `Workflow.encode`, which converts a front-end object into an `EncodingResult`. Similar wrappers then dispatch to homological algebra, invariants, and comparison routines. The `EncodingResult` object is important here because it carries not only the encoded mathematical object but also options, provenance, backend choice, and metadata.

Finally, several cross-cutting layers operate in parallel with the main algebraic pipeline. The data ingestion layer bridges raw data and formal presentations. The serialization layer makes front ends, encodings, and feature artifacts reproducible. The featurization layer treats encoded modules as reusable objects in statistical and machine-learning workflows. The visualization layer treats the same encoded objects as the basis for exploration and explanation. These are more than just auxiliary conveniences added after the mathematical design was fixed, they are

part of what it means for the encoded module to serve as a stable internal language across the whole system.

This layering does not follow the design of the flow-chart in figure 1, but ensures extensible software development and works in tandem to the conceptual user experience design. However, the rest of the thesis will follow the flow of figure 1 again (ultimately, that conceptualization of the library is the most expressive for users), with the next chapter describing pre-encoding objects and data interfaces.

5 Pre-encoding objects and data interfaces

Sections 2 and 3 introduced the mathematical reason that a finite encoding should exist and should be the object on which exact computation is carried out. Before one can build such an encoding, however, one needs a finite front end description of the module to be encoded. This chapter describes the principal front ends accepted by TAMER-OP.

The guiding principle is as follows: the library is intentionally permissive about the form in which a module is first described, but intentionally strict about the object on which later algebra is done. On the input side, one may begin with a module already indexed by a finite poset, with a $\mathbb{Z}^n$-presentation adapted to finitely determined modules, with a piecewise-linear presentation over $\mathbb{R}^n$, or with raw topological data from which such a presentation can be constructed. On the output side of the encoding step there is only the working language of a module on a finite encoding poset.

5.1 Finite-poset presentations: finite fringe presentations and indicator data

The simplest front end occurs when the indexing poset is already finite. In that case no geometric compression is needed to obtain a finite combinatorial object, and the basic question is only how to package the module in a form that is easy to manipulate and easy to compare with later encoded outputs.

Let P be a finite poset. As in Section 3, an upset $U \subseteq P$ determines an indicator module $\Bbbk\{U\}$, and a downset $D \subseteq P$ determines an indicator module $\Bbbk\{D\}$. Since P is finite, these indicator modules are themselves finite diagrams of finite-dimensional vector spaces.

Definition 20 (Finite fringe presentation on a finite poset)**.** Let P be a finite poset and let $\Bbbk$ be a field. A *finite fringe presentation* on P consists of finitely many upsets $U_1, \dots, U_m \subseteq P$, finitely many downsets $D_1, \dots, D_r \subseteq P$, and a morphism

$$\varphi : \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\},$$

with the associated module defined as

$$M := \operatorname{im}(\varphi).$$

Equivalently, once bases are chosen on the one-dimensional indicator summands, the presentation is specified by a matrix

$$\Phi \in \Bbbk^{r \times m}$$

subject to the support condition that $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$.

The matrix description is especially concrete on a finite poset. For each vertex $p \in P$, define the active column and row sets

$$I(p) = \{i \in \{1, \dots, m\} : p \in U_i\}, \qquad J(p) = \{j \in \{1, \dots, r\} : p \in D_j\}.$$

Evaluating the presentation at p yields a linear map

$$\varphi_p : \Bbbk^{I(p)} \longrightarrow \Bbbk^{J(p)}$$

whose matrix is the submatrix $\Phi_{J(p),I(p)}$. Thus the fiber at p is

$$M(p) = \operatorname{im}(\varphi_p), \qquad \dim_{\Bbbk} M(p) = \operatorname{rank}(\Phi_{J(p),I(p)}).$$

In other words, the module is read off from the same finite matrix together with the membership pattern of each vertex in the chosen upsets and downsets.

This front end matters for two central reasons. Firstly, it is the cleanest correctness model in the whole library. When the user already knows the finite indexing poset, there is no mystery with the encoding; all later kernels, cokernels, Hom, Ext, and invariant computations can be checked directly against this finite presentation. Secondly, finite fringe presentations are also the natural output of the encoding step, not just merely one among many input formats. The point of encoding is precisely to replace a module on a large poset by a finite-poset module or, equivalently, by a finite fringe presentation on a finite region poset. For that reason the finite fringe language is both a front end description and the simplest model of the finite encoding.

5.2 $\mathbb{Z}^n$ front end: flange presentations and finitely determined data

The next front end is adapted to modules over $\mathbb{Z}^n$ with the coordinatewise order. This is the natural discrete setting for many multifiltration constructions, and it is also the setting in which the connection with multigraded commutative algebra is most direct [CZ09]. A $\mathbb{Z}^n$-module is a functor from the poset category $\mathbb{Z}^n$ to vector spaces, equivalently a multigraded representation with structure maps in the positive coordinate directions.

A particularly important class is formed by finitely determined modules.

Definition 21 (Finitely determined $\mathbb{Z}^n$-module)**.** A $\mathbb{Z}^n$-module M is *finitely determined* if there exists $a \in \mathbb{Z}^n$ such that for every $b \geq a$ and every standard basis vector e_i, the structure map

$$M(b \leq b + e_i) : M(b) \longrightarrow M(b + e_i)$$

is an isomorphism.

This condition says that, sufficiently far in the positive orthant, the module stops changing in any essential way. It is the discrete analogue of eventual constancy on regions, and it is one of the standard hypotheses under which finite descriptions become possible.

Miller's theory packages these modules using *flats* and *injectives*. Fix a subset $\tau \subseteq \{1, \ldots, n\}$, thought of as a set of free coordinate directions, and a lattice point $b \in \mathbb{Z}^n$. The corresponding flat and injective regions are

$$F_{\tau,b} = \{a \in \mathbb{Z}^n : a_i \geq b_i \text{ for every } i \notin \tau\},$$
$$E_{\tau,b} = \{a \in \mathbb{Z}^n : a_i \leq b_i \text{ for every } i \notin \tau\}.$$

The coordinates in τ are unrestricted. Thus principal upsets and principal downsets occur as the special case $\tau = \varnothing$, while more general choices of τ allow one to model affine coordinate flats in the sense used in Miller's $\mathbb{Z}^n$-theory [Mil20].

Definition 22 (Flange presentation)**.** A *flange presentation* over $\mathbb{Z}^n$ consists of finitely many flats $F_1, \ldots, F_m$, finitely many injectives $E_1, \ldots, E_r$, and a matrix

$$\Phi \in \Bbbk^{r \times m}$$

with the support condition $\Phi_{ji} = 0$ whenever $F_i \cap E_j = \varnothing$. It presents the module

$$M := \operatorname{im}\left(\bigoplus_{i=1}^{m} \Bbbk\{F_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{E_j\} \right).$$

The local behavior is parallel to the finite fringe case. For a lattice point $g \in \mathbb{Z}^n$, let

$$I(g) = \{i : g \in F_i\}, \qquad J(g) = \{j : g \in E_j\}.$$

Then the fiber $M(g)$ is the image of the submatrix $\Phi_{J(g),I(g)}$, so fiber dimensions and local maps are determined by the active flats and injectives at g. What changes, compared to the finite-poset case, is that the ambient poset is infinite. The flange presentation therefore serves as a compact input language whose membership predicates can be evaluated anywhere in $\mathbb{Z}^n$, but which is intended to be encoded before more substantial algebra is done.

5.3 $\mathbb{R}^n$ front-end: PL fringes and box presentations

For modules over $\mathbb{R}^n$, one needs a geometric version of the same idea. The relevant inputs are piecewise-linear upsets and downsets, represented by finite polyhedral data. This is the correct front end for modules whose behavior changes along finitely many polyhedral boundaries, and it is also the natural setting for geometric constructions arising from stratified topology and sheaf-theoretic viewpoints on persistence [KS18, Mil23].

Definition 23 (PL upset and PL downset)**.** A *PL upset* in $\mathbb{R}^n$ is an upward-closed subset that is a finite union of convex polyhedra. A *PL downset* is defined dually as a downward-closed finite union of convex polyhedra.

The front end presentation is then the direct real-parameter analogue of a fringe presentation.

Definition 24 (PL fringe presentation)**.** A *PL fringe presentation* over $\mathbb{R}^n$ consists of finitely many PL upsets $U_1, \dots, U_m \subseteq \mathbb{R}^n$, finitely many PL downsets $D_1, \dots, D_r \subseteq \mathbb{R}^n$, and a matrix

$$\Phi \in \Bbbk^{r \times m}$$

with $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$. The associated module is again the image of the induced morphism

$$\bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\}.$$

At a point $x \in \mathbb{R}^n$, one evaluates the same active-set construction as before,

$$I(x) = \{i : x \in U_i\}, \qquad J(x) = \{j : x \in D_j\},$$

and the fiber is computed from the submatrix $\Phi_{J(x),I(x)}$. Thus the formal algebra is unchanged, with change happening in the geometry of membership and intersection. In the real setting, a correct implementation must be able to decide whether a point lies in one of the finitely many prescribed polyhedral regions and must be able to identify the induced constant regions of the presentation.

An important specialization is the axis-aligned box backend. Here one restricts to upsets and downsets of the form

$$U_\ell = \{x \in \mathbb{R}^n : x_i \geq \ell_i \text{ for all } i\}, \qquad D_u = \{x \in \mathbb{R}^n : x_i \leq u_i \text{ for all } i\}.$$

This format covers a large and practically relevant class of examples. Additionally, and importantly, they admit a much more direct encoding procedure, covered in the next section. So, ultimately, the polyhedral front end is mathematically more general, while the box front end is algorithmically much simpler, and the library supports both.

5.4 Raw data, filtrations, and graded complexes

Not every user begins with a presentation by upsets and downsets. In topological data analysis the more common starting point is raw data together with a rule for assigning parameter values to cells, simplices, edges, or vertices. For this reason TAMER-OP includes a data-ingestion layer whose role is to convert raw data into one of the presentation objects discussed above, usually by way of a graded complex.

Definition 25 (Graded complex)**.** Let K be a finite cell complex. A *graded complex* over a poset Q is the data of a grade

$$\deg(\sigma) \in Q$$

for each cell σ of K, subject to the monotonicity condition that if τ is a face of σ, then

$$\deg(\tau) \leq \deg(\sigma).$$

Equivalently, the chain groups of K become Q-graded and the boundary maps are degree-nonincreasing in the usual persistence sense.

From such a graded complex one obtains persistence modules by taking homology or cohomology degreewise. In the one-parameter setting this is the standard construction underlying persistent homology [ZC05]; in the multiparameter setting the same principle applies, but the grade now lies in $\mathbb{Z}^n$, $\mathbb{R}^n$, or another suitable poset.

The data-ingestion layer accepts several kinds of raw or partially structured input. The most basic data objects are point clouds, graphs, and images. In addition, the user may begin with an already constructed graded complex, a multicritical graded complex in which a cell carries several incomparable minimal grades, or a compressed simplex-tree representation of a multifiltered simplicial complex. The point is not that all of these objects are mathematically identical, but that each of them can be functorially transformed into a graded complex, from which an encoding can then be built.

The transformation is controlled by a typed filtration specification. For point clouds, representative families include Vietoris–Rips, density-modified Rips, landmark Rips, Delaunay-type, alpha-type, and related constructions. For graphs, the library supports lower-star, clique lower-star, edge-weighted, geodesic, centrality-based, and other graph-specific filtrations. For images and cubical data, lower-star and distance-based filtrations play the analogous role. The output of this stage is still pre-encoding, being a finite graded object with an explicitly recorded parameter assignment, not yet a finite encoding poset.

The next stage of the thesis is to explain how all these front ends we have discussed are converted into a finite encoding poset and a finite module on that poset.

6 Bibliography notes

References

[Mil20] E. Miller. *Homological algebra of modules over posets.* arXiv:2008.00063, 2020.

[Waa24] L. Waas. *Notes on abelianity of categories of finitely encodable persistence modules.* arXiv:2407.08666, 2024.

[Mil23] E. Miller. *Stratifications of real vector spaces from constructible sheaves with conical microsupport. Journal of Applied and Computational Topology*, 7(3):473–489, 2023. doi:10.1007/s41468-023-00112-1.

[KS18] M. Kashiwara and P. Schapira. *Persistent homology and microlocal sheaf theory. Journal of Applied and Computational Topology*, 2(1–2):83–113, 2018. arXiv:1705.00955.

[CB15] W. Crawley-Boevey. *Decomposition of pointwise finite-dimensional persistence modules*. *Journal of Algebra and Its Applications*, 14(5):1550066, 2015. doi:10.1142/S0219498815500668.

[ZC05] A. Zomorodian and G. Carlsson. *Computing persistent homology*. *Discrete & Computational Geometry*, 33:249–274, 2005. doi:10.1007/s00454-004-1146-y.

[CZ09] G. Carlsson and A. Zomorodian. *The theory of multidimensional persistence*. *Discrete & Computational Geometry*, 42:71–93, 2009. doi:10.1007/s00454-009-9176-0.

[CEH07] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. *Stability of persistence diagrams*. *Discrete & Computational Geometry*, 37(1):103–120, 2007. doi:10.1007/s00454-006-1276-5.

[ELZ02] H. Edelsbrunner, D. Letscher, and A. Zomorodian. *Topological persistence and simplification*. *Discrete & Computational Geometry*, 28:511–533, 2002. doi:10.1007/s00454-002-2885-2.

[Les15] M. Lesnick. *The theory of the interleaving distance on multidimensional persistence modules*. *Foundations of Computational Mathematics*, 15(3):613–650, 2015. doi:10.1007/s10208-015-9255-y.

[LW15] M. Lesnick and M. Wright. *Interactive visualization of 2-D persistence modules*. arXiv:1512.00180, 2015.

[RIVET] M. Lesnick and M. Wright. *RIVET: The rank invariant visualization and exploration tool* (software). `https://github.com/rivetTDA/rivet`.

[LS24] D. Loiseaux and H. Schreiber. *multipers: Multiparameter persistence for machine learning*. *Journal of Open Source Software*, 9(103):6773, 2024. doi:10.21105/joss.06773.

[CDSGO16] F. Chazal, V. de Silva, M. Glisse, and S. Y. Oudot. *The Structure and Stability of Persistence Modules*. SpringerBriefs in Mathematics. Springer, 2016. doi:10.1007/978-3-319-42545-0.

[Oud15] S. Y. Oudot. *Persistence Theory: From Quiver Representations to Data Analysis*. Mathematical Surveys and Monographs, Vol. 209. American Mathematical Society, 2015.

[EH10] H. Edelsbrunner and J. L. Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

[Mac98] S. Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics, Vol. 5. Springer, 2nd edition, 1998. doi:10.1007/978-1-4757-4721-8.

[Wei94] C. A. Weibel. *An Introduction to Homological Algebra*. Cambridge Studies in Advanced Mathematics, Vol. 38. Cambridge University Press, 1994.

[Eis95] D. Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*. Graduate Texts in Mathematics, Vol. 150. Springer, 1995. doi:10.1007/978-1-4612-5350-1.

[MS05] E. Miller and B. Sturmfels. *Combinatorial Commutative Algebra*. Graduate Texts in Mathematics, Vol. 227. Springer, 2005. doi:10.1007/b138602.